

Documento Técnico – Sistema de Órdenes de Servicio

1. Diagnóstico inicial

Durante el análisis del sistema propuesto para la gestión de órdenes de servicio, técnicos y clientes, se identificaron varios puntos críticos relacionados con arquitectura, seguridad y diseño de datos.

1.1 Arquitectura general

La solución mezcla responsabilidades entre capas (UI, lógica de negocio y acceso a datos).

Falta una separación clara entre:

- Controllers
- Services
- Data Access (Dapper repositories)

Riesgo: código difícil de mantener y escalar.

1.2 Acceso a datos

Uso obligatorio de Dapper, pero sin estrategia definida de consultas dinámicas.

Posible tendencia a queries duplicadas o hardcodeadas.

Riesgo:

baja reutilización

alto acoplamiento con SQL

1.3 Validaciones

Validaciones de negocio no centralizadas.

Posible dependencia del frontend para reglas críticas (ej: estado de orden, existencia de técnico/cliente).

Riesgo: inconsistencias de datos

vulnerabilidad a manipulación desde cliente

1.4 Seguridad

Autenticación JWT sin estrategia de roles definida.

Password storage requiere hash seguro (BCrypt recomendado).

Riesgo: autenticación débil si no se normaliza el hashing

1.5 Base de datos

Falta de índices en campos usados para filtros.

Dependencia fuerte de joins sin optimización.

Riesgo: degradación de rendimiento con crecimiento de datos

2. Propuesta de refactorización

2.1 Arquitectura propuesta (Clean + Dapper)

Separar en capas claras:

/Controllers

/Services

/Repositories (Dapper)

/Domain (Models)

/Infrastructure (DB connection)

2.2 Patrón de acceso a datos

Implementar repositorios con Dapper:

```
public interface IServiceOrderRepository
{
    Task<IEnumerable<ServiceOrderDto>> GetAllAsync();
    Task<ServiceOrder> GetByIdAsync(int id);
    Task CreateAsync(ServiceOrder order);
    Task UpdateAsync(ServiceOrder order);
    Task DeleteAsync(int id);
}
```

Beneficio: desacopla SQL del controlador, mejora de estabilidad.

2.3 Consultas dinámicas (filtros)

Implementar construcción dinámica de SQL:

```
var sql = new StringBuilder(baseQuery);

if (!string.IsNullOrEmpty(status))
    sql.Append(" AND so.status = @status");

if (!string.IsNullOrEmpty(technician))
    sql.Append(" AND t.specialty ILIKE @technician");
```

Beneficio: evita duplicación de queries, escalable a múltiples filtros combinables.

2.4 Seguridad

Implementar BCrypt para hashing de contraseñas:

```
BCrypt.Net.BCrypt.HashPassword(password);
```

Validación JWT en middleware

Posible extensión: roles (Admin / User)

2.5 Optimización de base de datos

Agregar índices estratégicos:

```
CREATE INDEX idx_service_orders_status ON service_orders(status);
```

```
CREATE INDEX idx_service_orders_created_at ON service_orders(created_at);
```

```
CREATE INDEX idx_clients_document ON clients(document_number);
```

Beneficio: mejora rendimiento en filtros combinados

2.6 Validación de negocio en backend

Reglas centralizadas:

- No crear orden si cliente no existe
- No asignar técnico inexistente
- Validar estados permitidos

3. Justificación de decisiones

3.1 Uso de Dapper

Se eligió Dapper en lugar de Entity Framework porque:

- Permite control total de SQL
- Mejora rendimiento en consultas complejas
- Es ideal para filtros dinámicos avanzados

3.2 Separación por capas

Se adopta arquitectura limpia porque:

- facilita mantenimiento
- permite escalar el sistema
- mejora testabilidad
- reduce acoplamiento

3.3 SQL manual (sin LINQ)

Se evita LINQ en consultas de órdenes porque:

- el requerimiento exige SQL explícito
- permite optimización directa en PostgreSQL/Oracle
- mejora control de joins y filtros

3.4 JWT + BCrypt

Se implementa seguridad estándar porque:

- JWT permite stateless authentication
- BCrypt evita exposición de contraseñas
- estándar en sistemas empresariales modernos

3.5 Índices en base de datos

Se agregan índices porque:

- el sistema depende de filtros combinados
- el crecimiento de datos afectará rendimiento sin optimización

Conclusión

La propuesta mejora significativamente:

1. escalabilidad del sistema
2. seguridad de autenticación
3. rendimiento de consultas
4. mantenibilidad del código
5. Además, alinea la solución con estándares reales de desarrollo empresarial usando .NET + Dapper + PostgreSQL.